Git Overview:

It is important to update your own branch periodically while working on it. Make sure to push any changes you make to your local branch to your own branch at the end of each session. When you completed a task and are sure there won't be a merge conflict, make a pull request to merge your changes into main. If you have any questions, feel free to contact Kaitlyn (if you don't have my number it is 906 280 0343)

How to create your own branch

1. Before creating a new branch make sure you are current with main.

   ```
   git switch main
   git pull origin main
   ```

2. Create your new branch and switch to it.

   ```
   git switch -c yourname_branch
   ```

3. Confirm you are on the branch.

   ```
   git branch
   ```

4. Push your new branch to GitHub

   ```
   git push -u origin yourname_branch
   ```

5. After the first push you can just use push

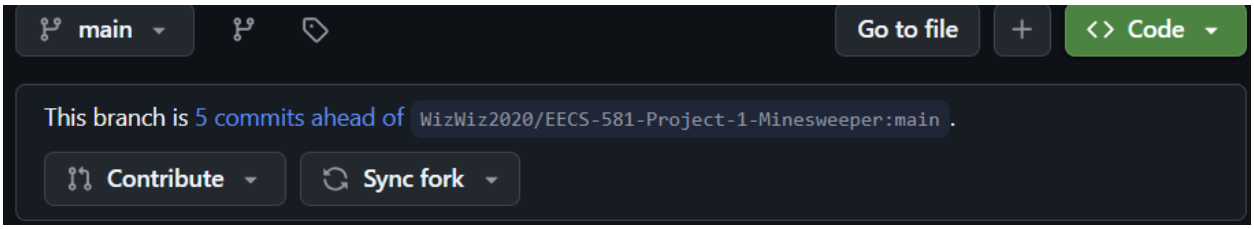
   ```
   git push
   ```

Update your own branch from main

Before pushing any changes to main, make sure your branch is current with main. Also, never push updates directly to main.

How to check if you are current with Main on a forked repository.

The repository will compare your branch with Main from the original repo, not the one we are using. The command line will have to be used to check our main instead.

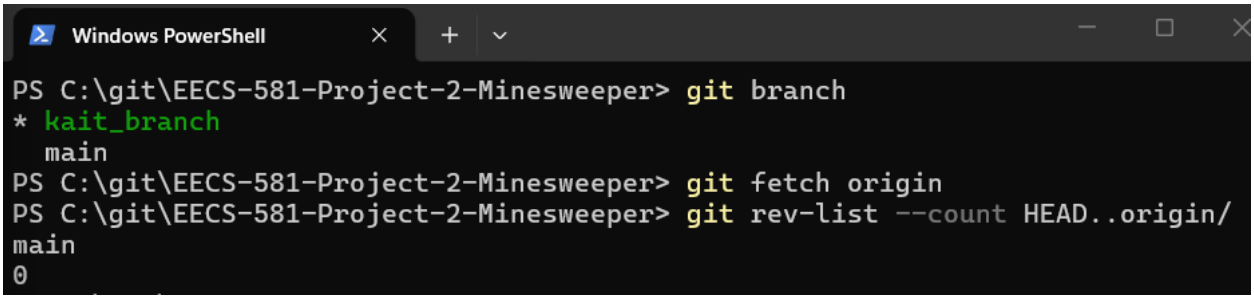


Type these commands in to confirm your branch is up to date with main. If the terminal displays 0, your branch has everything currently on main. If it displays something greater than 0, your branch is missing that many commits from Main.

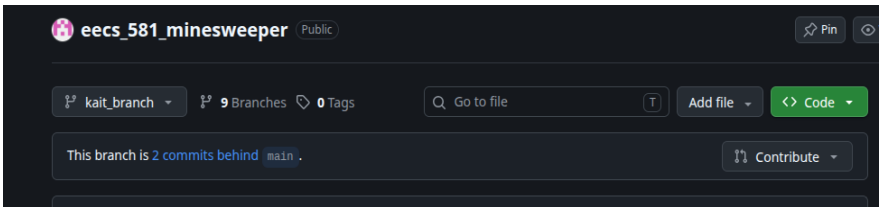
```
git branch
```

```
git fetch origin
```

```
git rev-list --count HEAD..origin/main
```



1. Check your branch on GitHub



2. My branch is two commits behind main, so I need to pull the changes down to my branch on my computer.

   Open the terminal where your minesweeper folder is and type git status to see what branch you are on. git status will also allow you to see any changes you made locally on your computer

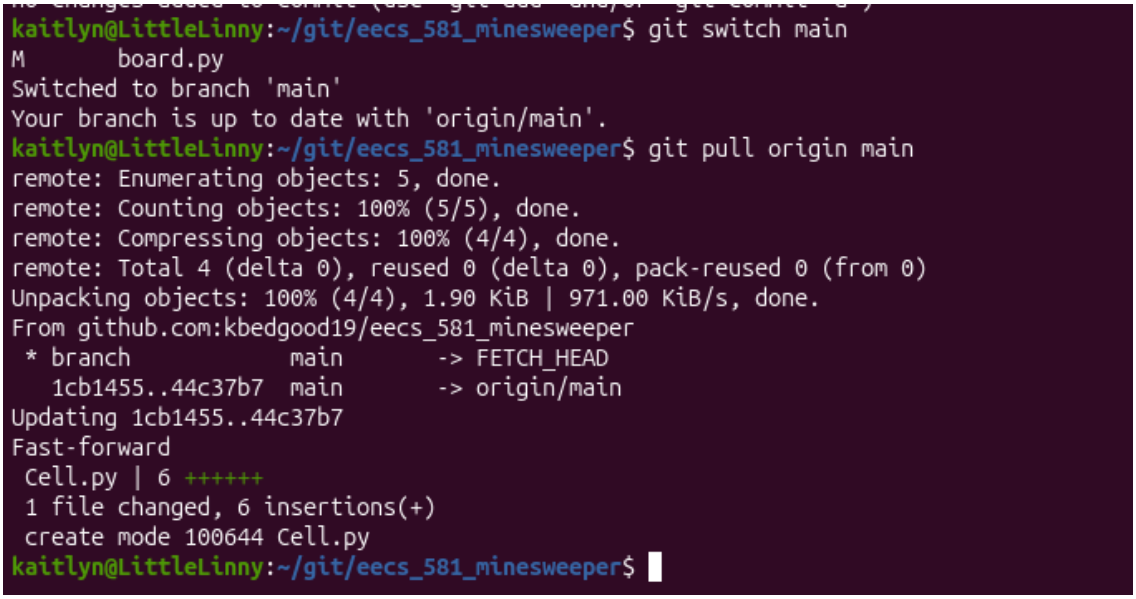
   ```
   git status
   ```



3. Switch to the main branch to pull down any changes made to main onto your local branch.

Type in these commands to switch to main and pull from main.

```
git switch main
git pull origin main
```

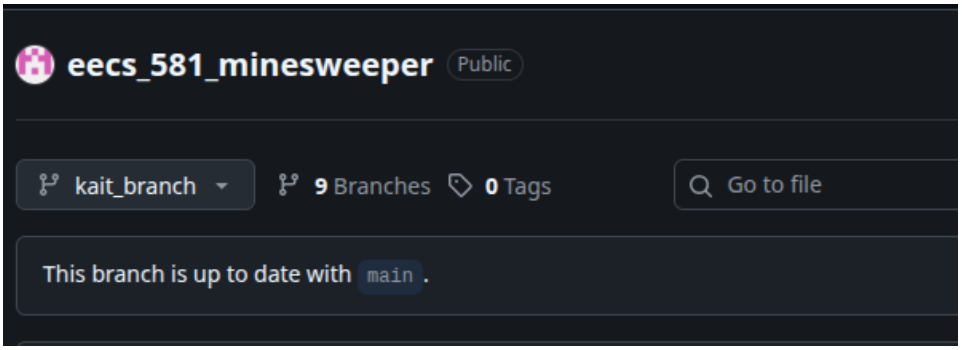


4. Your local branch is now updated with main, now update your branch on GitHub. Switch to your own branch, merge your branch with main, and push your updated branch to GitHub.

```
git switch your_branch
git merge main
git push -u origin YOUR_BRANCH_NAME <---use this first time can
                                      just use git push after
```

```
 create mode 100644 Cell.py
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git switch kait_branch
M       board.py
Switched to branch 'kait_branch'
Your branch is up to date with 'origin/kait_branch'.
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git merge main
Updating 1cb1455..44c37b7
Fast-forward
 Cell.py | 6 ++++++
 1 file changed, 6 insertions(+)
 create mode 100644 Cell.py
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git push
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:kbedgood19/eecs_581_minesweeper.git
   1cb1455..44c37b7  kait_branch -> kait_branch
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$
```

5. Check GitHub to make sure changes have been updated



Push to your own branch

Push to your own branch after updating the program locally on your own machine.

1. Open a terminal within the mine sweeper folder. git status can be used to see what branch you are on and the changes you have made.

```
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git status
On branch kait_branch
Your branch is up to date with 'origin/kait_branch'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   board.py

no changes added to commit (use "git add" and/or "git commit -a")
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$
```

`git status`

`git switch your_branch` <--------- only use if not on your own branch

2. Use git add to include the files you want to commit but does not perform the actual commit. Use "`git add .`" if you want to add all changes (if you use this, it won't affect files you have not changed) or use "`git add example.py`" if you want to only add changes done to a specific file. After git add, nothing will appear in the terminal, if you want to make sure the changes have been made, use "`git status`"

```
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git add .
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git status
On branch kait_branch
Your branch is up to date with 'origin/kait_branch'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   board.py

kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$
```

`git add .`

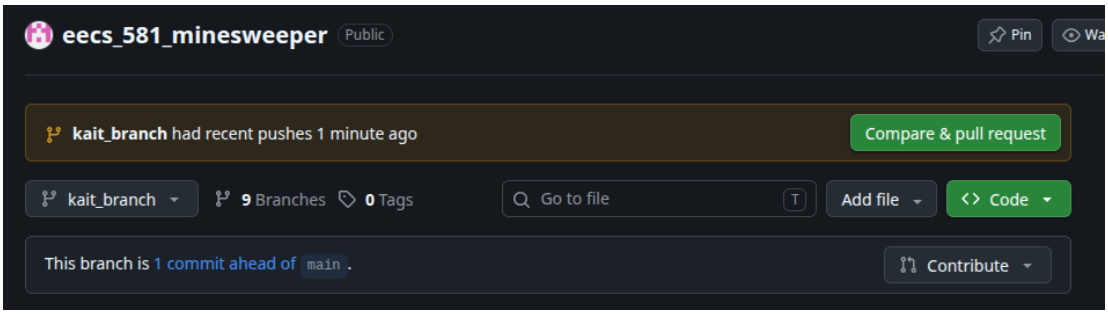
3. Use "`git commit –m "comments about the code"` "to describe the updates made, cannot push without committing. Then push changes to GitHub.

```
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git commit -m "Added labels to board.py"
[kait_branch f3b1fb9] Added labels to board.py
 1 file changed, 26 insertions(+), 8 deletions(-)
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 596 bytes | 596.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:kbedgood19/eecs_581_minesweeper.git
   44c37b7..f3b1fb9  kait_branch -> kait_branch
kaitlyn@LittleLinny:~/git/eecs_581_minesweeper$
```

```
git commit -m "Describe your changes"

git push
```

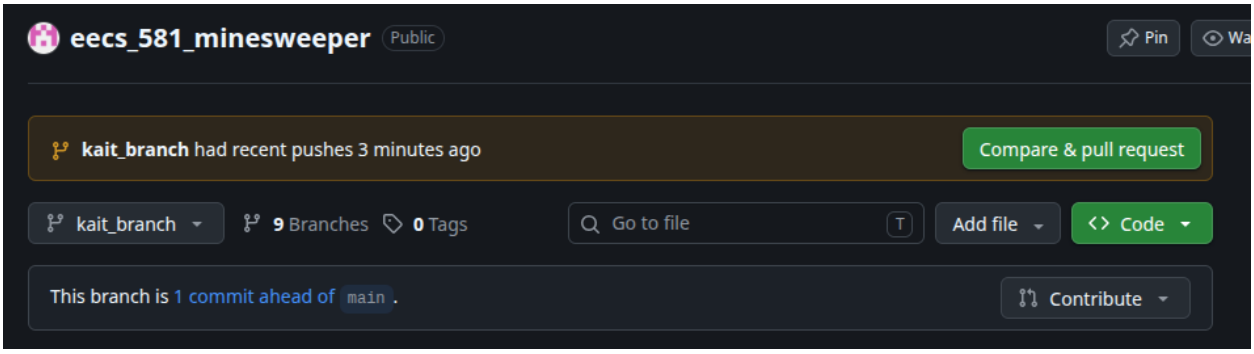
4. Check on GitHub that changes have been pushed



Update main with your work

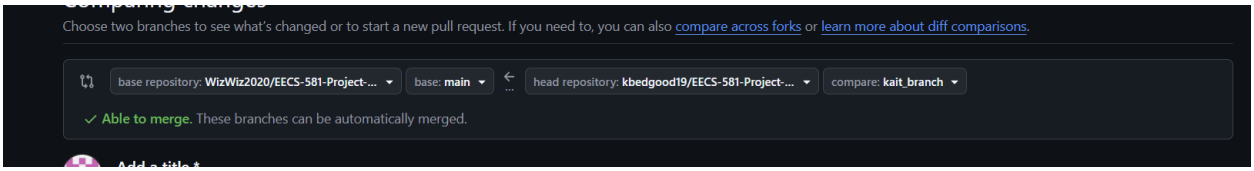
After updating your own branch and you are ready to update main with your code.

1. Go to GitHub on your own branch, you will see an option to make a pull request.

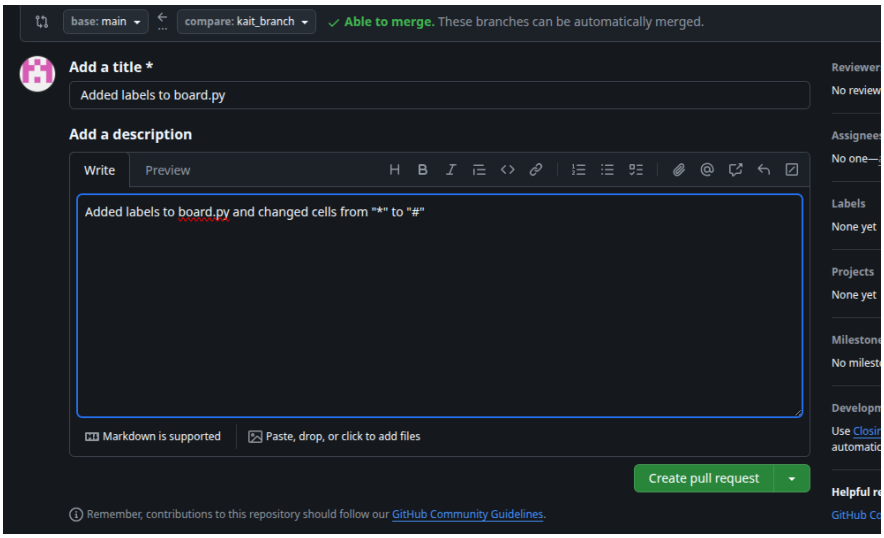


If you are working off a forked repository:

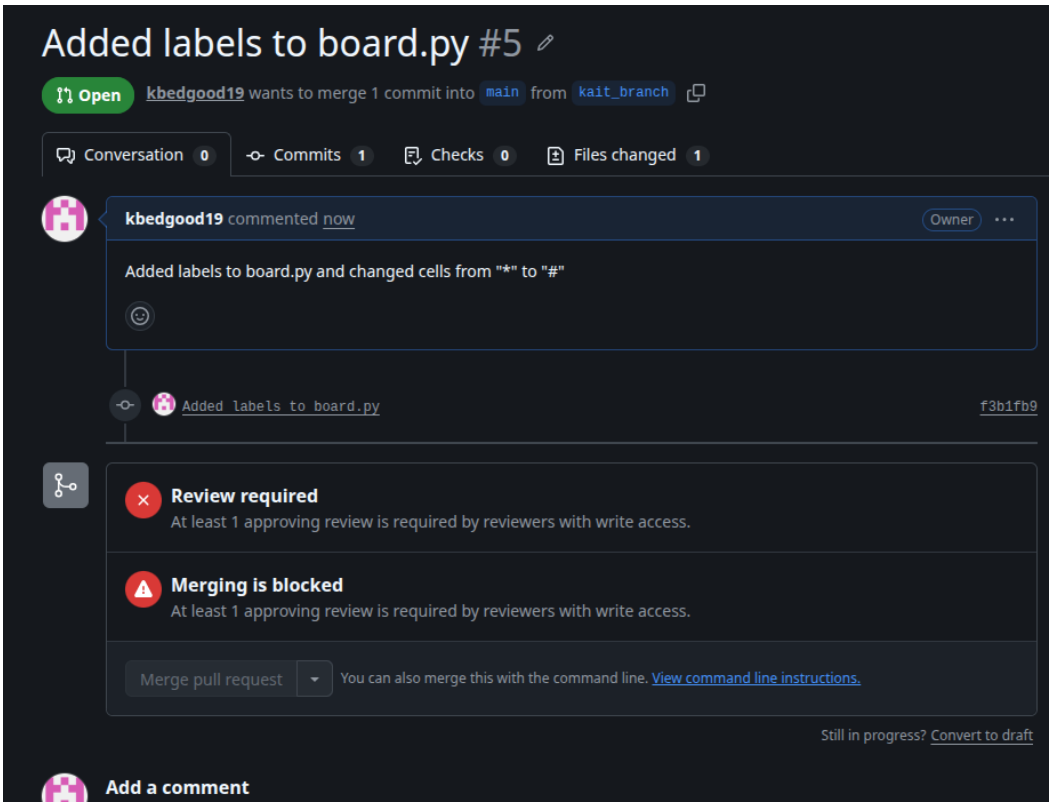
There is a drop-down menu that allows you to choose which Main to use for the pull request. Make sure to change the base repository drop-down menu to kbedgood19.



2. Click on Compare & pull request and write any notes about your code that would be helpful



3. Click on Create pull request, you will not be able to merge your code into main. Another contributor has to review your code and approve the merge. This helps prevent conflicts on main. Once done, you can contact another group member to review the merge or just wait (I will get an email, but I might not see it right away)



Complete Pull Request

After another group mate does a pull request, another group member has to review the code and approve the pull request.

NOTE: If you are unsure of the changes made by someone else or believe there will be a merge conflict, contact Kaitlyn and the person who made the request.

If you are working off a forked repository:

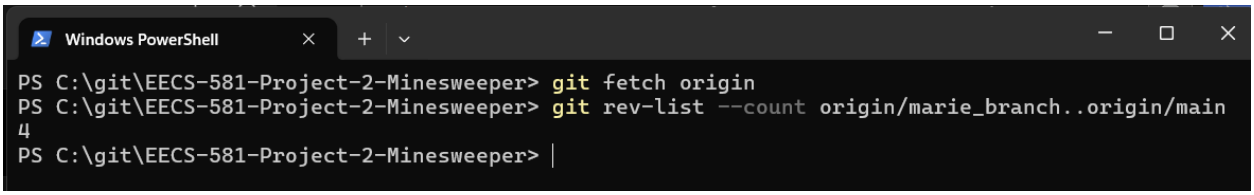
You will need to use the command line to make sure the branch the person doing the pull request is up to date with our Main. Since the repository is a fork, GitHub only shows how many commits our branches are ahead of or behind the original repository's Main.

The commands will print 0 if the branch is not missing any commits from our Main. If the branch is behind our Main, it will print the amount of commits it is behind.

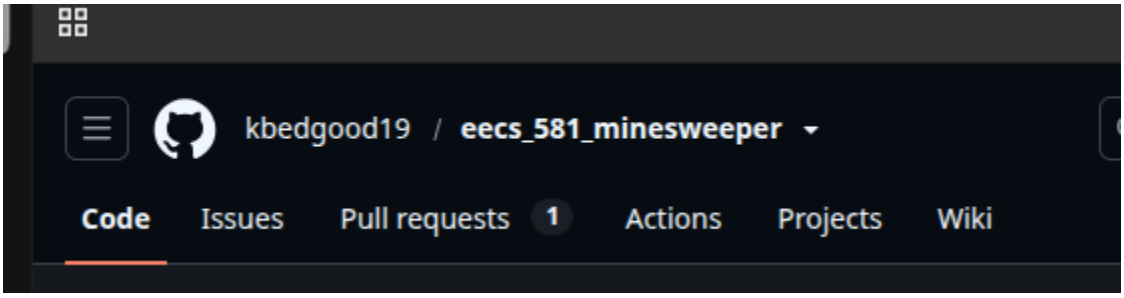
Commands to run:

```
git fetch origin
```

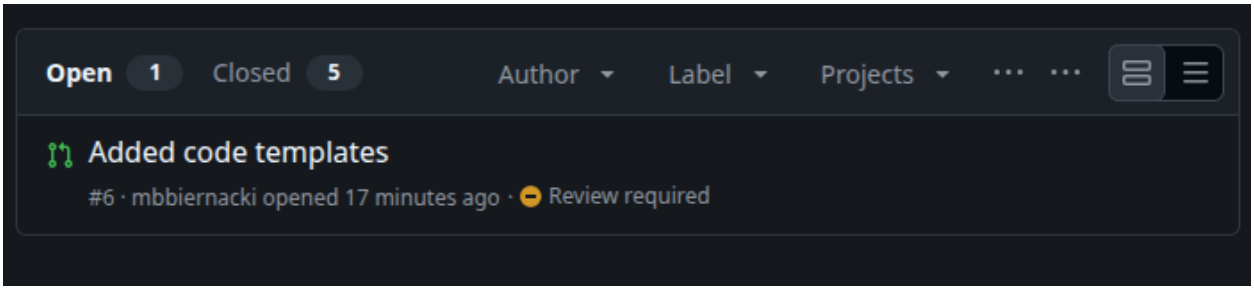
```
git rev-list --count origin/BRANCH_NAME..origin/main
```



1. Click on Pull Request on GitHub (if there is a pull request there should be a number by it)

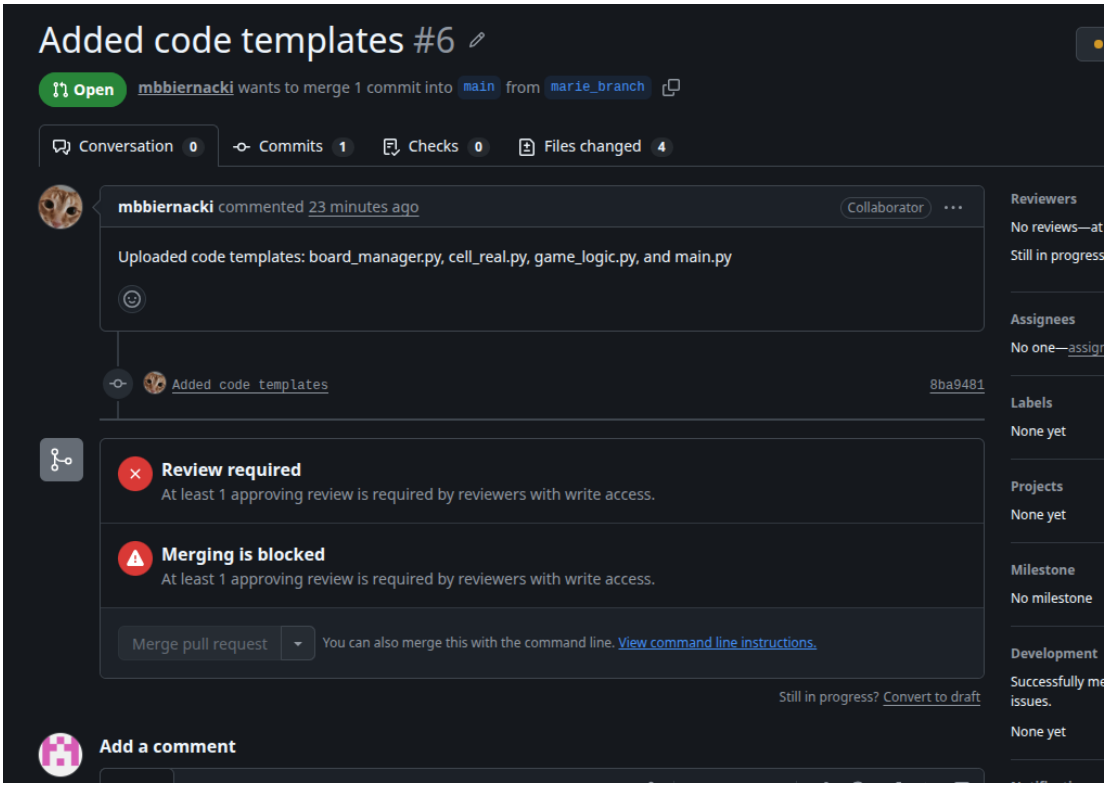


2. Click on the pull request. If you move your mouse over the request, it will give you an option to open it.

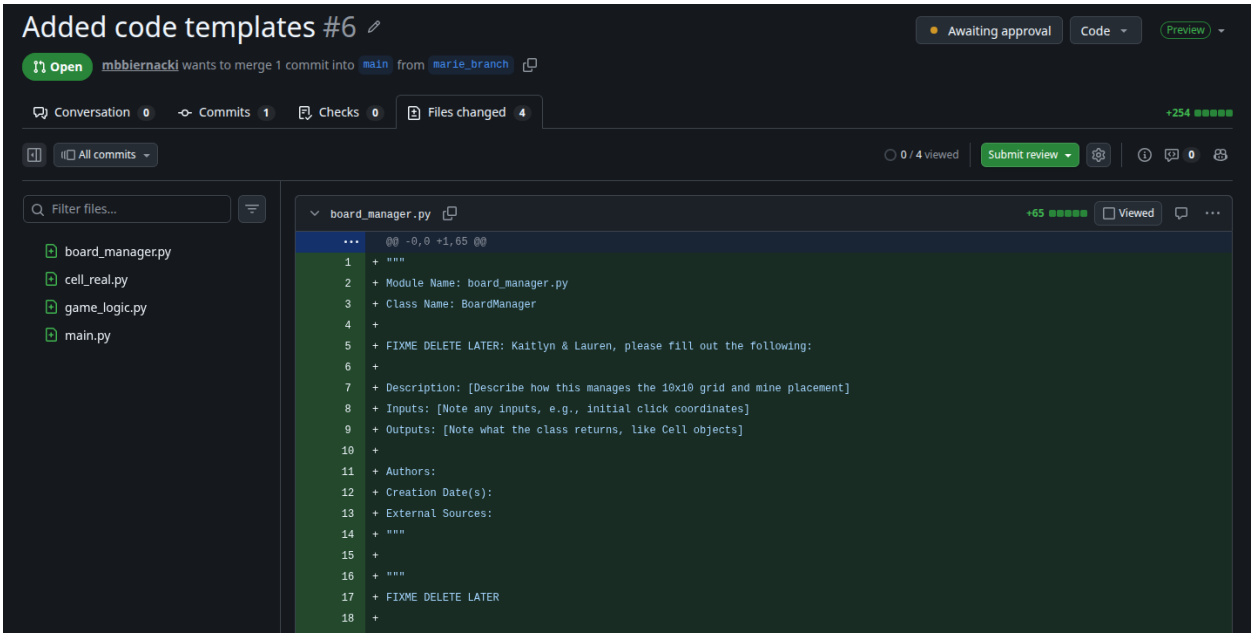




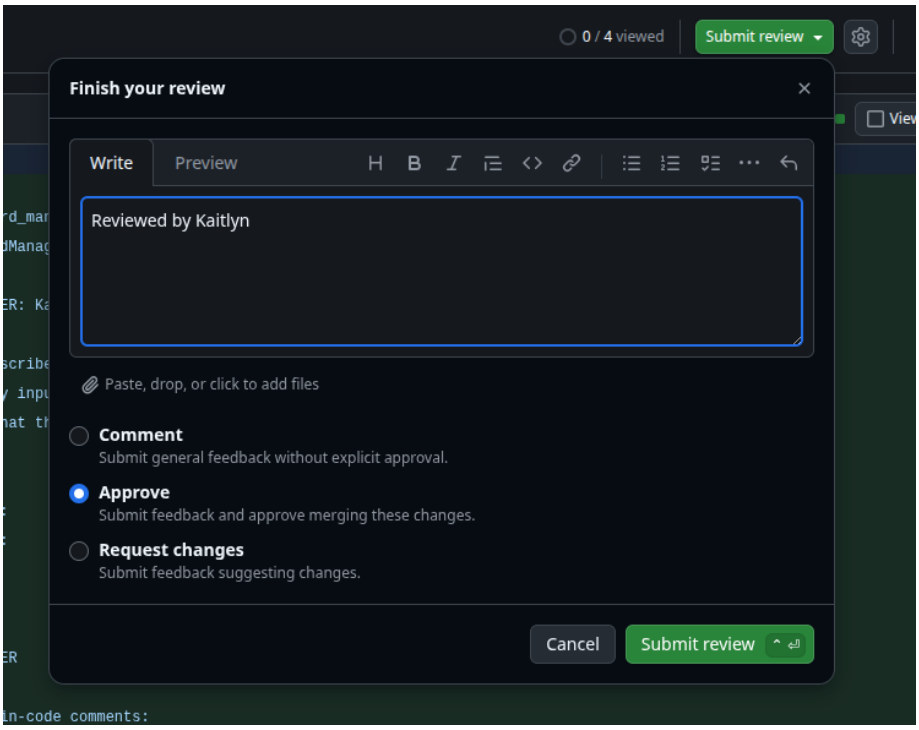
3. After clicking on the pull request, it will bring you to a screen with comments the requester has made about their updates. Click on files changed.



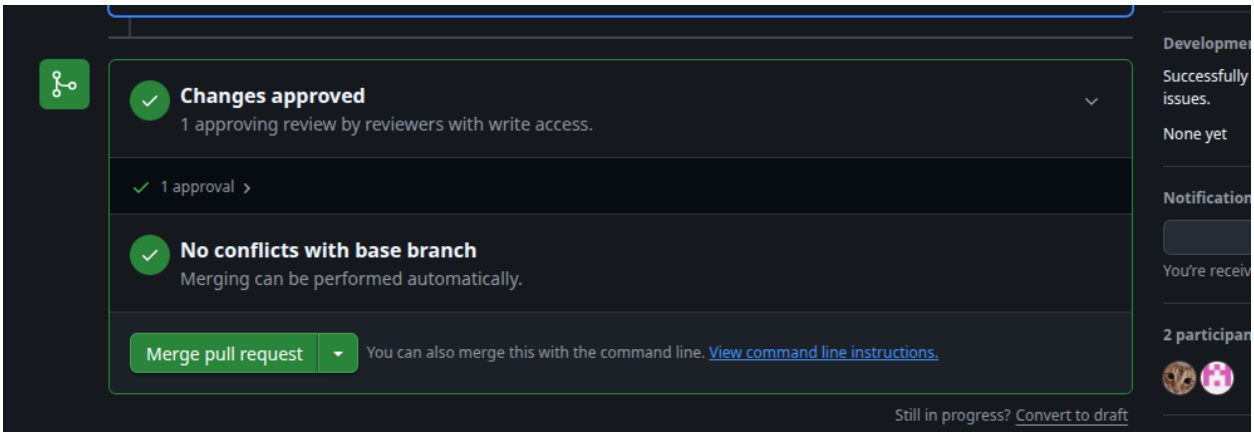
4. Here you can see all changes made. The files created and code added will be in green and anything removed will be in red. Please look at all code changes to make sure nothing important is being removed. After reviewing the code ,click submit review.



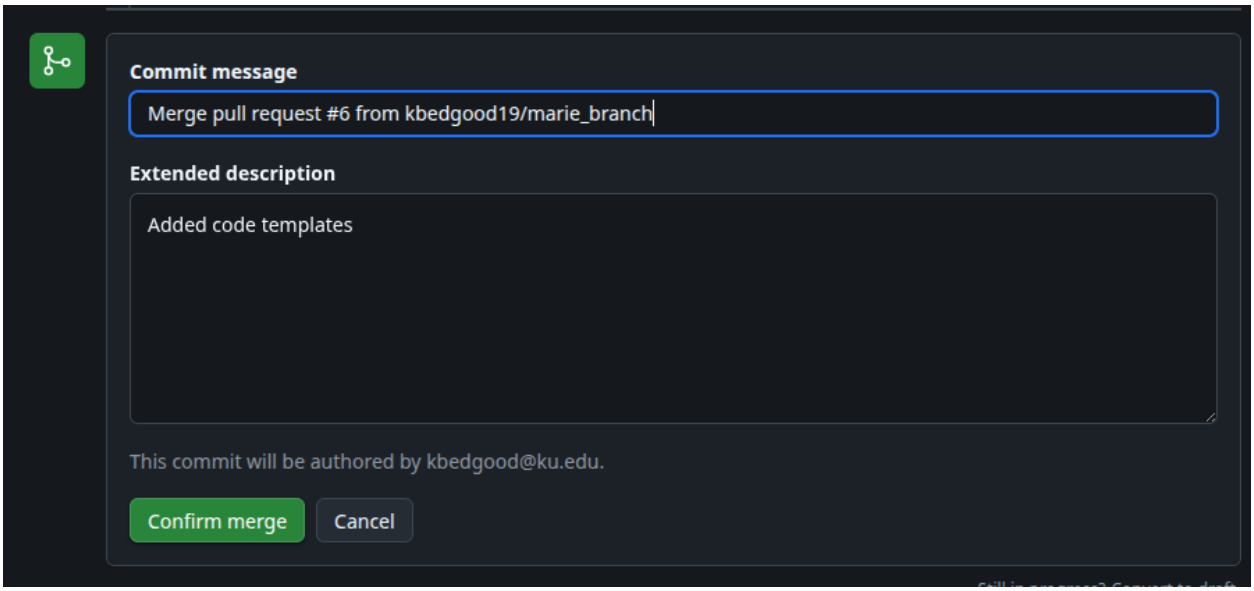
5. Write any comments you have in the comment box and select approve. Then click submit review.



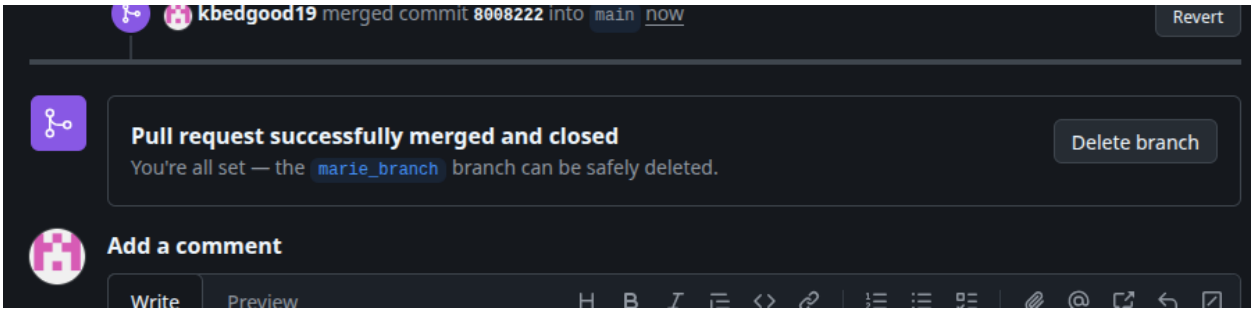
6. Click merge pull request



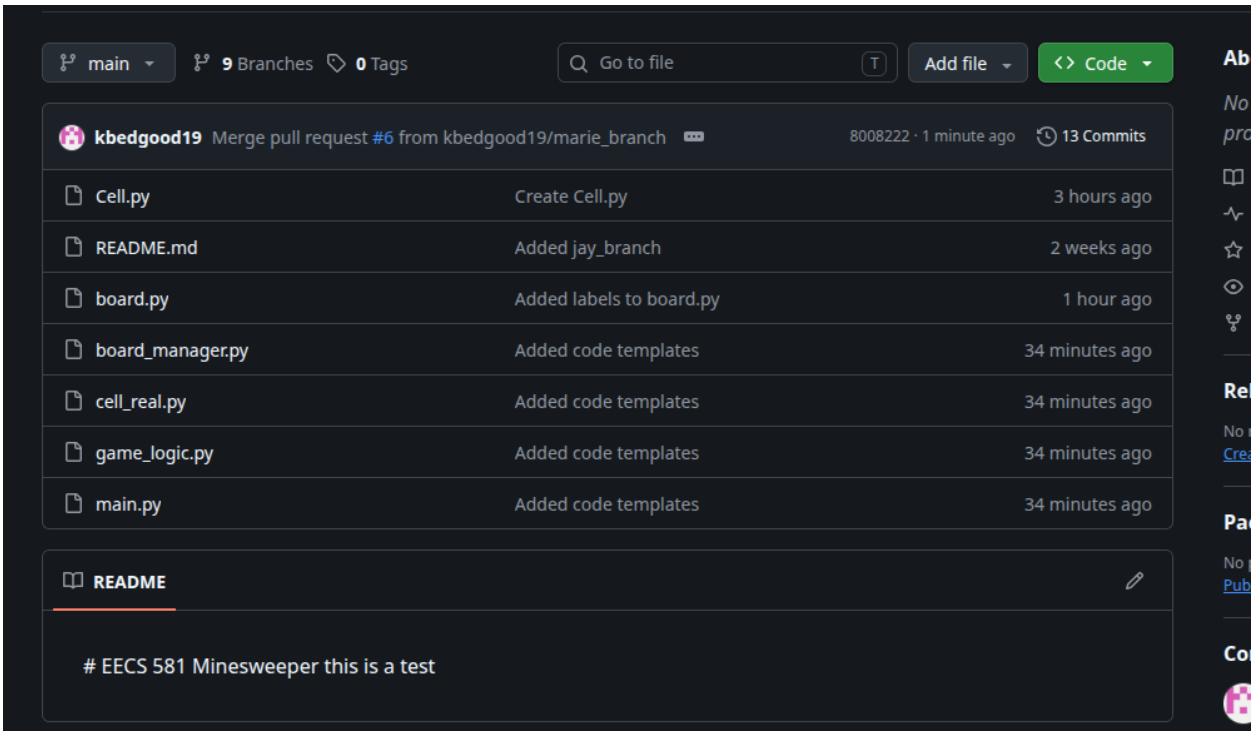
7. Click confirm merge



8. Merge request is now closed



9. Go to the main branch and confirm changes



10. Make sure to update your own branch so you are current with main.